

# Claude Code - Learner Prompt Packet

Every prompt taught in the course, copy-paste ready. Built 2026-07-16 from the delivered decks.

## Day 1 - Fundamentals

---

### The Agentic Loop & Setup - Paste this: first contact

*Use it for: What you are practicing*

```
claude auth login
```

```
/doctor
```

```
Read package.json and tell me what this project does in three sentences.
```

### Memory & CLAUDE.md - Paste this: memory workflow

*Use it for: What you are practicing*

```
/init
```

```
Remember: we use pnpm, not npm. Save that so you never suggest npm again.
```

```
Write HANDOFF.md: what you tried, what worked, what failed, what is next - so a fresh session can continue from that file alone.
```

### Context Mastery - Paste this: sharp delegation

*Use it for: What you are practicing*

```
Add JWT auth to @auth.js following the middleware pattern in @payments.js. Run the auth tests to prove it.
```

```
POST /api/tasks returns 500. Read @src/routes/tasks.js, find the cause, fix only that, and show me the diff.
```

### Commands, Cost & Permissions - Build a human-invoked skill

*Use it for: What you are practicing*

```
Create .claude/skills/review/SKILL.md with name review, description 'Review a target for security, performance, and style', disable-model-invocation: true, allowed-tools Read, Grep, Glob; body: review $ARGUMENTS and return prioritized findings with file references.
```

```
/review src/auth/login.ts
```

## Day 2 - Extensions

---

### Skills - Paste this: your first skill

*Use it for: What you are practicing*

Create a skill at `.claude/skills/code-review/SKILL.md` in the current project directory. Trigger on 'review the code', 'check for bugs', 'QA this file'. Review for security, error handling, and test coverage. Read-only tools.

Show me the SKILL.md you created and explain what each frontmatter field does.

## Hooks & Custom Commands - Paste this: a /test-this command

*Use it for: What you are practicing*

Create `.claude/skills/test-this/SKILL.md` with name test-this, description 'Generate tests for a file', `disable-model-invocation: true`; body: 'Write unit tests for \$ARGUMENTS. Cover happy path, edge cases, and error handling. Run the tests and report results.'

```
/test-this src/utils/helpers.js
```

## Advanced Skill Systems - Paste this: eval your skill

*Use it for: What you are practicing*

Create an eval set for my code-review skill: 5 prompts that SHOULD trigger it and 3 that should NOT. Run each, report which fired correctly as a pass/fail table.

Analyze the failures and propose one improved description. Show old vs new side by side.

## Plugins & Marketplaces - Paste this: package your toolkit

*Use it for: What you are practicing*

Run `claude plugin init` to scaffold a plugin called my-toolkit. Move my `.claude/skills`, `hooks`, and `commands` into it and update the manifest to reference them.

Install my-toolkit from the local path into a fresh test project and list what became available, with namespaces.

## Day 3 - Orchestration & Production

### MCP: connect to everything - Paste this: wire up a server

*Use it for: What you are practicing*

```
claude mcp add github -- npx -y @modelcontextprotocol/server-github
```

```
claude mcp add --transport http linear https://mcp.linear.app/mcp
```

```
claude mcp login linear
```

### Subagents & agent teams - Paste this: nest and grade

*Use it for: What you are practicing*

Launch a research lead subagent for the payments module. Have it spawn one subagent per service, then compile a single dependency report for me.

Spawn 3 subagents to each draft the retry logic independently. Then spawn a grader subagent that scores all 3 against rubric.md and reports the winner with scores.

## Production & CI/CD - Paste this: headless runs

*Use it for: What you are practicing*

```
claude -p "Review src/ for error handling gaps" --max-turns 5 --allowedTools
"Read,Grep,Glob"
```

```
claude -p "List all TODOs with file paths" --output-format json
```

```
git diff main...HEAD | claude -p "Review this diff. Flag critical issues only."
```

## Scale, cost & new surfaces - Paste this: parallel worktrees

*Use it for: What you are practicing*

```
git worktree add ../app-feature-a feature-a
```

```
git worktree add ../app-bugfix-b bugfix-b
```

```
cd ../app-feature-a && claude # session 1 - repeat in b for session 2
```